

ExploraSO

Explorador de Serviços do Sistema Operacional

Aplicação acadêmica em Python 3 • Sistemas Operacionais



Vamos falar sobre...



O que é o ExploraSO?

Uma aplicação de terminal criada para demonstrar, de forma prática, como programas utilizam serviços fornecidos pelo Sistema Operacional.

O projeto prioriza simplicidade, leitura do código e demonstração em sala — sem frameworks, sem banco de dados e sem dependências externas.

Python 3

Terminal / CLI

Biblioteca padrão

6 operações

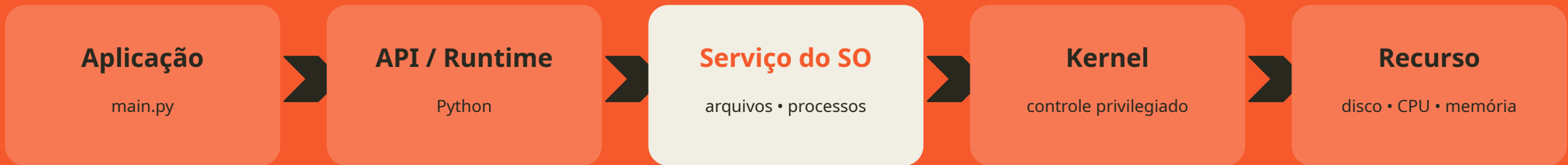
4 categorias

0 dependências
externas

Validado no Windows
com Python 3.13

Da aplicação ao kernel

O ponto principal não é “chamar o hardware”, mas solicitar serviços ao SO através de interfaces.



Atenção ao conceito

Funções como `open()`, `read()`, `write()`, `os.getpid()` e `subprocess.Popen()` são APIs de alto nível. Elas não devem ser confundidas automaticamente com uma System Call específica.

O que a aplicação faz

Seis operações distribuídas em quatro categorias didáticas de serviços do SO.



Arquivos

Criar arquivo
Escrever e ler

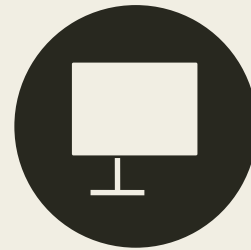
Opção 1-2



Diretórios

Criar pasta
Listar conteúdo

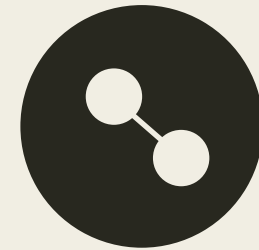
Opção 3



Sistema

SO • release
arquitetura • Python

Opção 4



Processos

Consultar PID
Criar filho e aguardar

Opção 5-6

Como funciona na prática

O menu permite testar cada serviço isoladamente ou executar a demonstração completa.

```
> python main.py
```

```
=== ExploraSO ===  
[1] Criar arquivo  
[2] Escrever e ler  
[3] Diretórios  
[4] Informações do sistema  
[5] Mostrar PID  
[6] Processo filho  
[7] Demonstração completa  
[0] Sair
```

Escolha: 7

Demonstração completa

1

Arquivos e diretórios

cria • grava • lê • lista

2

Informações do sistema

SO • arquitetura • runtime

3

Processos

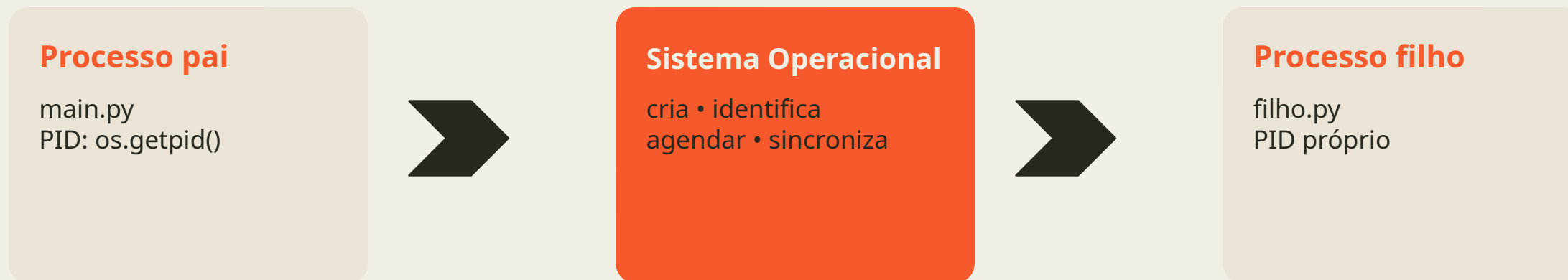
PID • processo filho • wait()

~2 s de espera

repetível e seguro

Processos: o ponto mais importante

A opção 6 mostra, de forma visual, o ciclo de criação e sincronização entre um processo pai e um processo filho.



APIs usadas

`subprocess.Popen()`

`Popen.pid`

`Popen.wait()`

`sys.executable`

PID = Process Identifier • `wait()` faz o pai aguardar o término do filho • a ordem de algumas mensagens pode variar pelo escalonamento.

Perguntas que podem cair

Quatro respostas curtas para fechar a apresentação com segurança.

1. O que foi implementado?

Uma aplicação CLI que demonstra seis operações em quatro categorias de serviços do SO.

2. Qual API/função foi usada?

Path/open/read/write, platform, os.getpid() e subprocess.Popen()/wait().

3. Qual serviço do SO está envolvido?

Sistema de arquivos, diretórios, informações do ambiente e gerenciamento de processos.

4. Como demonstrar?

Execute python main.py e escolha 7 para percorrer todas as operações em sequência.

Obrigado!

Dúvidas ou perguntas?